Revisão Python: Gestão de memória, Tipos de Dados (abstrato, opaco, estruturas genéricas) e TADs

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza
Créditos: Prof. Zanoni Dias

Revisão de Python

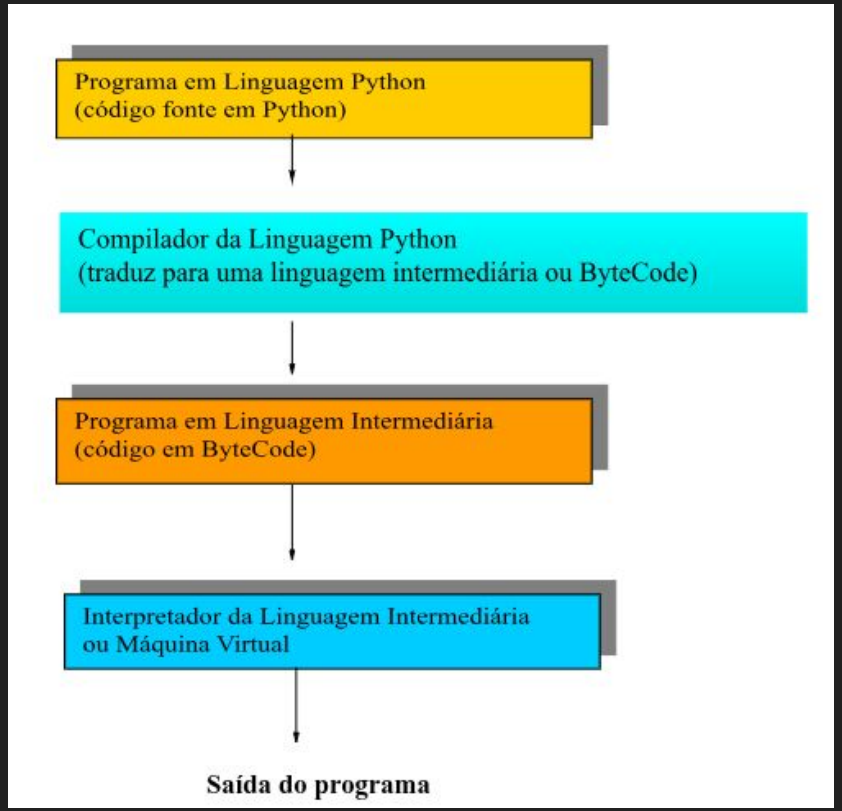
- Python é uma linguagem interpretada, ou seja, o “compilador” Python traduz o programa fonte para uma linguagem intermediária (em geral chamada de ByteCode) que é interpretada por um programa interpretador, popularmente conhecido como Máquina Virtual.

- Um programa interpretado é menos eficiente que um programa compilado.

- Apresenta portabilidade (roda em qualquer ambiente que tenha a Máquina. Virtual).

Revisão de Python

- Abstraindo a linguagem intermediária, o Python é simplesmente interpretado.

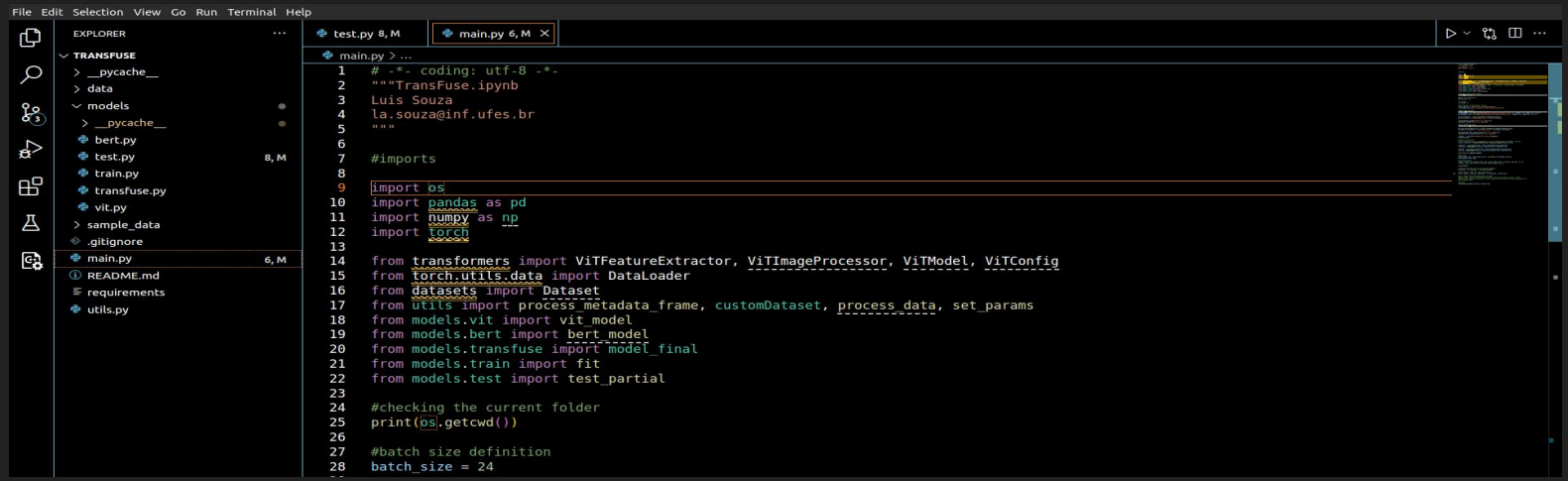


Desenvolvimento Python

- Linha de comando (terminal)

```
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900
32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a = 32
>>> b = 15
>>> c = a + b
>>> c
47
>>> print(c)
47
>>> a + b
47
```

- IDE (Integrated Development Environment) (VSCode)



Classes da linguagem Python

- Python é orientado a objetos
- Objetos são agrupados em classes pré-existentes (intrínsecas ou built-in) ou podem ser criadas por um programa (objetos customizados).

Classe	Descrição	Objeto imutável?
int	número inteiro	sim
float	número em ponto flutuante	sim
bool	booleano	sim
listas	sequência de objetos	não
tuplas	sequência de objetos	sim
str	cadeia de caracteres	sim
set	sequência não ordenada de objetos	não
frozenset	sequência não ordenada de objetos	sim
dict	dicionário associativo	não

Classes da linguagem Python

- Python é orientada a objetos
- Objetos são agrupados em classes pré-existentes (intrínsecas ou built-in) ou podem ser criadas por um programa (objetos customizados).

Funcionam como estruturas genéricas em Python

```
struct nome_s
{
    int var1;
    float var2;
    char var3;
}
```

Classe	Descrição	Objeto imutável?
int	número inteiro	sim
float	número em ponto flutuante	sim
bool	booleano	sim
listas	sequência de objetos	não
tuplas	sequência de objetos	sim
str	cadeia de caracteres	sim
set	sequência não ordenada de objetos	não
frozenset	sequência não ordenada de objetos	sim
dict	dicionário associativo	não

Variáveis e Tipos (dinâmicos)

- Variáveis contém endereço de memória no qual objeto está associado (similar C++) - a função **id()** devolve o endereço de memória do objeto.
- Podem ter qualquer combinação de letras, dígitos (exceto começo) e underline, e não podem ser iguais as palavras reservadas da linguagem.

```
False      as         continue   else       from       in
not        return     yield      None       assert     def
except     global     is         or         try        True
break      del        finally    if         lambda     pass
while      and        class      elif       for        import
nonlocal   raise      with
```

- Tipos são dinâmicos:

```
# x é um int
x = 1
print(type(x))
# agora x é um str
x = "abcd"
print(type(x))
```

A classe *set*

- Permite a representação e manipulação de conjuntos como na álgebra, com elementos entre {} (exceto conjunto vazio, que usa o construtor da classe “s1 = set()”.
- *Sets* são diferentes de listas - no primeiro não existem elementos repetidos e nem ordenação de elementos (não permite indexação).

```
s1 = {1, 2, 3}
s2 = {2, 3, 4, 7}
print(s1 | s2) # união
print(s1 & s2) # intersecção
# pertinência
for i in range(5): print(i, i in s1)
```

A classe *dict*

- Permite a criação de chaves customizadas dentro de uma lista, associando valores a estas chaves.

```
# associa strings a números
ds = {"branco":23, "preto":107, "vermelho":-234, "amarelo":31}
for st in ds: print(st)
for st in ds: print(ds[st])

# alterando e aumentando ds
ds["amarelo"] = -31
ds["azul"] = 1739        # com atribuição a novo elemento
ds.update({"preto":108, "violeta":-46}) # usando o método update
print(ds)
```

Expressões e Operadores

- Operadores e operandos são combinados a fim de criar expressões aritméticas, lógicas. A semântica depende dos operandos.
- Operadores lógicos: *not, and, or*
  - Operam sobre variáveis e retornam True ou False.
- Operadores de igualdade: *is, is not, ==, !=*
  - Operam sobre variáveis e retornam True ou False.
- Operadores de bits (bitwise): *~ (complemento), & (and), | (or), ^ (XOR), << (deslocamento esquerdo), >> (deslocamento direito)*
  - Operam sobre variáveis inteiras e retornam suas representações em bits.

Expressões e Operadores

- Operadores sobre sequências (listas, strings, tuplas): sejam s e t de qualquer um dos tipos supracitados
  - Sequências admitem indexação e passo negativos e operações de comparação.

Operação	Ação
s[j]	Acesso ao elemento j da sequência
s[inicio:fim]	Acesso a uma fatia destes elementos
s[inicio:fim:passo]	Acesso a uma fatia com deslocamento
s + t	Concatenção de s e t
k * s	O mesmo que s + s + s + ... (k vezes)
v in s	True se valor v é um dos elementos da sequência
v not in s	True se valor v não for um dos elementos da sequência

Expressões e Operadores

- Operadores de remoção de elementos em listas (remove, del, pop)
  - A existência do elemento na lista (ou não) pode gerar erros de execução.

```
# remove - remove o primeiro elemento igual da lista
a = [0, 2, 1, 2, 3, 2]
a.remove(2)
print(a)

# del - remove um elemento específico - não retorna
a = [0, 2, 1, 2, 3, 2]
del a[2]
print(a)

# pop - remove o elemento de um índice específico - retorna
a = [0, 2, 1, 2, 3, 2]
a.pop(2)
print(a)
```

Precedência de Operadores

Operator Precedence		
	Type	**Symbols**
1	member access	expr.member
2	function/method calls container subscripts/slices	expr(...) expr[...]
3	exponentiation	**
4	unary operators	+expr, −expr, ~expr
5	multiplication, division	*, /, //, %
6	addition, subtraction	+, −
7	bitwise shifting	<<, >>
8	bitwise-and	&
9	bitwise-xor	^
10	bitwise-or	|
11	comparisons containment	**is**, **is not**, ==, !=, <, <=, >, >= **in**, **not in**
12	logical-not	**not** expr
13	logical-and	**and**
14	logical-or	**or**
15	conditional	val1 **if** cond **else** val2
16	assignments	=, +=, −=, *=, etc.

Data Structures and Algorithms in Python – Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser

Funções

- Definição:

  def nome_da_funcao(<parametros>):

  <corpo da funcao>

  return <valor>

- Funções podem receber ou não parâmetros, e podem ou não retornar valores (int, float, bool, lista, tupla). Se o **return** não for especificado, a função retorna **None** por *default*.
- Chamada:

  …nome_da_funcao(<parametros_atuais>)

***IMPORTANTE:** parâmetros são passados, por padrão, por “atribuição” para funções em Python.

Funções

- Parâmetros passados por referência: listas, sets, dicts. Alterações dentro da função são refletidas fora dela.

```
# zera uma lista
def zera(vt):
  for i in range(len(vt)):
        vt[i] = 0

vs = [1, 2, 3, 4]
zera(vs)
# vs voltará da função com valor [0, 0, 0, 0]
```

Funções

- Parâmetros podem ter valores default, e sua passagem pode se tornar facultativa.

```
def func(a, b=15, c=-1):
```

- Também, pode-se definir explicitamente o valor de um parâmetro utilizando sua definição na assinatura da função.

```
func(c = 33):
```

Expressão *lambda*

- Podem resumir a utilização de uma função empregando uma sintaxe mais matemática.

```
somaquad = lambda x, y: x*x + y*y
def somaquad(x, y):
    return x*x + y*y
```

- Podem ser empregadas no lugar de funções.

```
ar = [1, 2, 3, 4, 5, 6]
ssq = list(map(lambda x: 1 + x + x * x, ar))
print (ssq)

ssq = list(filter(lambda x: x % 2 == 0, ar))
print (ssq)
```

Operações padrão da Linguagem

Common Built-In Functions	
Calling Syntax	**Description**
abs(x)	Return the absolute value of a number.
all(iterable)	Return True if bool(e) is True for each element e.
any(iterable)	Return True if bool(e) is True for at least one element e.
chr(integer)	Return a one-character string with the given Unicode code point.
divmod(x, y)	Return (x // y, x % y) as tuple, if x and y are integers.
hash(obj)	Return an integer hash value for the object (see Chapter 10).
id(obj)	Return the unique integer serving as an "identity" for the object.
input(prompt)	Return a string from standard input; the prompt is optional.
isinstance(obj, cls)	Determine if obj is an instance of the class (or a subclass).
iter(iterable)	Return a new iterator object for the parameter (see Section 1.8).
len(iterable)	Return the number of elements in the given iteration.
map(f, iter1, iter2, ...)	Return an iterator yielding the result of function calls f(e1, e2, ...) for respective elements e1 ∈ iter1, e2 ∈ iter2, ...
max(iterable)	Return the largest element of the given iteration.
max(a, b, c, ...)	Return the largest of the arguments.
min(iterable)	Return the smallest element of the given iteration.
min(a, b, c, ...)	Return the smallest of the arguments.
next(iterator)	Return the next element reported by the iterator (see Section 1.8).
open(filename, mode)	Open a file with the given name and access mode.
ord(char)	Return the Unicode code point of the given character.
pow(x, y)	Return the value x^y (as an integer if x and y are integers); equivalent to x ** y.
pow(x, y, z)	Return the value (x^y mod z) as an integer.
print(obj1, obj2, ...)	Print the arguments, with separating spaces and trailing newline.
range(stop)	Construct an iteration of values 0, 1, ..., stop − 1.
range(start, stop)	Construct an iteration of values start, start + 1, ..., stop − 1.
range(start, stop, step)	Construct an iteration of values start, start + step, start + 2*step, ...
reversed(sequence)	Return an iteration of the sequence in reverse.
round(x)	Return the nearest int value (a tie is broken toward the even value).
round(x, k)	Return the value rounded to the nearest 10^{-k} (return-type matches x).
sorted(iterable)	Return a list containing elements of the iterable in sorted order.
sum(iterable)	Return the sum of the elements in the iterable (must be numeric).
type(obj)	Return the class to which the instance obj belongs.

Data Structures and Algorithms in Python – Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser.

Tratamento de Exceções

- Erros inesperados podem ocorrer durante a execução de programas, que podem ser tratados pelo interpretador a fim de não interromper a execução ou não retornar valores inconsistentes ao usuário.
- Abaixo, as principais exceções tratáveis pelo Python:

Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax obj.foo, if obj has no member named foo
EOFError	Raised if "end of file" reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by next(iterator) if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., sqrt(−5))
ZeroDivisionError	Raised when any division operator used with 0 as divisor

Tratamento de Exceções

- Interceptação de exceção:

```
try:
   ratio = x / y
except ZeroDivisionError:
   print("Tentativa de divisão por zero")

# abrir o arquivo para leitura
try:
   arq = open(NomeArq, "r")
except:
   print("Arquivo não pode ser aberto")
```

Tratamento de Exceções

- Interceptação de exceção:

```
# loop até ler um float válido
while True:
    try:
        val = float(input("Entre com um float:"))
    except:
        # Entra neste ponto em caso de qualquer exceção.
        # Em particular quando algo errado é digitado.
        print("Entrada inválida")
    else:
        # Entra neste ponto se a digitação for correta
        print("Entrada válida")
        break
```

Tratamento de Exceções

- Aplicação de exceções no tratamento do processamento de uma função:

```
from math import sqrt
def minha_sqrt(x):
   if x < 0:
       raise ValueError('O parâmetro não pode ser negativo')
   return sqrt(x)

print("teste 1 - tratando a exceção")
try:
   print(minha_sqrt(-2))
except:
   print("Erro - parâmetro inválido")

print("teste 2 - sem tratar a exceção")
print(minha_sqrt(-1))
```

Tratamento de Exceções

- Aplicação de exceções no tratamento do processamento de uma função:

Cláusula raise abandona a função!

```
from math import sqrt
def minha_sqrt(x):
    if x < 0:
        raise ValueError('O parâmetro não pode ser negativo')
    return sqrt(x)

print("teste 1 - tratando a exceção")
try:
    print(minha_sqrt(-2))
except:
    print("Erro - parâmetro inválido")

print("teste 2 - sem tratar a exceção")
print(minha_sqrt(-1))
```

```
teste 1 - tratando a exceção
Erro - parâmetro inválido
teste 2 - sem tratar a exceção
Traceback (most recent call last):
  File "C:/... /teste exceptions e mensagens.py", line 14, in <module>
    print(minha_sqrt(-1))
  File "C:/.../teste exceptions e mensagens.py", line 4, in minha_sqrt
    raise ValueError('O parâmetro não pode ser negativo')
ValueError: O parâmetro não pode ser negativo
```

Iterators e Generators

- Caracteriza-se como “iterabilidade” a operações de varrer um objeto que seja percorrível de forma sequencial do início ao fim (listas, dicts, tuplas, sets e strings).
- Exemplo:

```
s = [45, 25, -12]
t = (1, -4, 82, 0)
c = {1, 4, -8, 120}
st = "exemplo"
d = {"x":21, "y":-78, "chato":1248}
# k é um iterador que varre os iteráveis
for k in s: print(k, " ", end = "")
print()
for k in t: print(k, " ", end = "")
print()
for k in c: print(k, " ", end = "")
print()
for k in st: print(k, " ", end = "")
print()
for k in d: print(k, " ", end = "")
print()
for k in d: print(d[k], " ", end = "")
print()
```

Iterators e Generators

- Geradores são objetos iteráveis criados a partir de funções. O comando *yield* retorna, a cada iteração, um valor que pode ser posteriormente iterado (não confundir com *return*).

```
# gerador dos números de fibonacci menores ou iguais a n
def fib(n):
   a = 0
   b = 1
   while a <= n:
       yield a
       a, b = b, a + b
# imprime os números de fibonacci menores ou iguais a 100
for x in fib(100): print(x)
# gera todos os anos bissextos do ano a1 até o ano a2
def bissexto(a1, a2):
   for k in range(a1, a2 + 1):
       if (k % 4 == 0 and k % 100 != 0) or k % 400 == 0: yield k
# imprime os anos bissextos de 1980 a 2025
for x in bissexto(1980, 2025): print(x)
```

Sintaxe Resumida para Loops

```
# vetor de n elementos com zeros
v = [0 for k in range(n)]
# matriz de n x m elementos zeros
mat = [m * [0] for k in range(n)]
# todos os fatores de N
fat = [k for k in range(2, N) if N % k == 0]

# set dos quadrados de 0 a t
ss = {j * j for j in range(t + 1)}
# dic dos quadrados de 0 a t
dd = {j:j * j for j in range(t + 1)}

# gerador dos quadrados de 0 a t-1
for k in (j * j for j in range(t)): print(k)
```

*Importante: o mesmo resultado pode ser obtido usando explicitamente o for-loop.

Modules e o comando *Import*

- Além das mais de 140 funções disponíveis nas versões *core* do Python, outras podem ser importadas de módulos. Por exemplo, mais funções matemáticas podem ser utilizadas importando o módulo math.

- Definições

from <module> import f1, f2, …

import module

- **Módulos são implementações com a extensão .py. Módulos customizados podem ser importados, enquanto que módulos já desenvolvidos, em sua maioria, precisam ser instalados utilizando a instrução pip.**

Modules importantes que fazem parte da instalação básica do Python

Existing Modules	
Module Name	**Description**
array	Provides compact array storage for primitive types.
collections	Defines additional data structures and abstract base classes involving collections of objects.
copy	Defines general functions for making copies of objects.
heapq	Provides heap-based priority queue functions (see Section 9.3.7).
math	Defines common mathematical constants and functions.
os	Provides support for interactions with the operating system.
random	Provides random number generation.
re	Provides support for processing regular expressions.
sys	Provides additional level of interaction with the Python interpreter.
time	Provides support for measuring time, or delaying a program.

Modules importantes não básicos

- **NumPy:** funções matemáticas usadas em operações com matrizes e vetores multidimensionais.
- **MatPlotLib:** funções para traçar gráficos.
- **SciPy:** funções matemáticas, computação científica e engenharia (interpolação de funções, otimização, transformadas de Fourier, processamento de sinais, etc).
- **Pandas:** funções para análise e visualização de dados. Possibilita o carregamento e salvamento de dados tabulares, aleḿ de fornecer ferramentas para proporcionar sua correlação.
- **Scikit-learn:** funções especializadas em processamento de dados e construção de modelos de inteligência artificial.

Tipos Abstratos de Dados (TAD)

- Na linguagem Python, os tipos de dados são dinâmicos, estabelecidos em tempo de execução. Este comportamento já proporciona a reutilização de variáveis para a utilização de diversos problemas (mas outros podem surgir).

- De linguagens clássicas e menos sofisticadas de implementação (como C), existe o conceito de tipo abstrato de dados, onde são definidos objetos específicos e operações que manipulam estes objetos. É de responsabilidade do desenvolvedor implementar os objetos e os métodos que operam sobre estes objetos, ficando para o usuário somente sua utilização.

- A interface define o comportamento das operações implementadas para o objeto.



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr